WILLIAM & MARY

CHARTERED 1693

AI for S/W Eng

Prof. Antonio Mastropaolo

Training a Transformer model for Predicting `if` statements

Your initial objective is creating two training datasets (*i.e.,* pre-training and fine-tuning) required to train a Transformer model capable of automatically recommending suitable `if` statements in Python functions.

Pre-training: You are required to pre-train a Transformer model using one of the pre-training objective explained during class (*i.e.,* MLM, CLM *etc.*). For example, if you select a T5-based model, then during pre-training you randomly masks 15% of the model's input (in our case, a Python function) and ask the model to guess them. This helps with learning patterns and structure of the programming langauge. Note that tokenization (and masking) must be performed using a trained tokenizer. This means, that first you have to train a tokenizer on your pre-training corpus and then you can use it for pre-training and fine-tuning. **Using an already pre-trained tokenizer (*i.e.,* huggingface's tokenizers) is not allowed.** You can explore pre-trained Transformer solutions also (*i.e.,* CodeT5+, CodeBERT *etc.*) hosted on huggingface, however in that case you are required to further pre-train the model by designing an appropriate pre-training objective that you assume help the model *e.g., LANCE Multi-objective pre-training.*. The alternative is pre-training a DL model from scratch.

Fine-tuning: After pre-training the model, you will keep moving with fine-tuning for the specifc task of interest (*i.e.,* recommending `if` conditions). In this case, the model will take as input a function having a special token masking a single `if` condition, and will try to guess it.

How many instances do we need? We recommend to have at least 150k instances for pre-training and 50k for fine-tuning, where an instance a pair made of a Python function with one $if_{statement}$ masked and the output, would be the masked statement. The more data, the better – but try to use your time efficiently, especially when it comes to training. Remember to set aside 10% of your fine-tuning dataset as a validation set (for example, to decide when to stop training or which hyperparameters to use) and another 10% as a test set. On Blackboard, you can find an additional test set which we will use to compare the performance of the different approaches. We will then create a ranking from the best- to the least-performing models, and in class we will discuss how specific design choices may have influenced each model's performance – both positively and negatively.

Where to get the instances? GitHub is your friend!. Focus only on Python projects, clone and parse their latest snapshot to extract the functions. For the projects' selection, consider using `https://seart-ghs.si.usi.ch`. **DataHub (`https://seart-dh.si.usi.ch`) for this project is not allowed**

As discussed in class, the quality of the training data is of paramount importance. Therefore, it's crucial that you implement all strategies necessary to ensure high data quality. Additionally, consider whether you will provide any contextual information to the model during pre-training and/or fine-tuning, such as code tokens or AST representations. Will you include very short functions in the training set? How will you handle functions

that don't contain `if` statements? These are the strategies you need to design, and we expect well-reasoned justifications for your choices.

Submission Instructions

Deadline: Tuesday October 24th, 2025 @12:00 PM.

Each student must submit on Blackboard, in the assignment section created specifically for Assignment 1 - the link to their GitHub repository, which should include:

1. A 3-page document describing the dataset construction process and the model training procedure and evaluation. The document it concludes with a brief discussion of the achieved results.
2. For each test set (*i.e.,* the one you built and the one provided), create a prediction `csv` file with the following columns:
   - Input provided to the model;
   - Whether the prediction is correct (`true/false`);
   - Expected `if` condition;
   - Predicted `if` condition;
   - Prediction score (`0-100`);

   Name the `csv` files as `generated-testset.csv`, and `provided-testset.csv`.